

Déclencheurs/Triggers (1/13)

Définition

- Les déclencheurs ou triggers sont des séquences d'actions définies par le programmeur qui se déclenchent, non pas sur un appel, mais directement quand un événement particulier (spécifié lors de la définition du trigger) sur une ou plusieurs tables se produit.
- Un trigger sera un objet stocké (comme une table ou une procédure)

Principe

- Un trigger se lance automatiquement lorsqu'un événement se produit.
- Par événement, on entend toute modification des données se trouvant dans les tables.
- On s'en sert pour contrôler ou appliquer des contraintes qu'il est impossible de formuler de façon déclarative.

Déclencheurs/Triggers (2/13)

Evénements déclencheurs

Lors de la création d'un trigger, il convient de préciser quel est le type d'événement qui le déclenche:

- *Insert*
- *Delete*
- *Update, ...*

Moments d'exécution

Il faut également préciser si le trigger doit être exécuté avant (BEFORE) ou après (AFTER) l'événement.

Durée

L'action associée à un trigger est un bloc PL/SQL enregistré dans la base.

Un trigger est opérationnel jusqu'à la suppression de la table à laquelle il est lié.

Le nom du trigger doit être unique dans la base de données

Déclencheurs/Triggers (3/13)

Types de triggers

Les triggers lignes (row trigger) sont exécutés **séparément** pour chaque ligne modifiée dans la table.

– *Ils sont très utiles s'il faut mesurer une évolution pour certaines valeurs, effectuer des opérations pour chaque ligne en question.*

Les triggers de table (trigger global/statement trigger) sont exécutés **une seule fois lorsque des** modifications surviennent sur une table (même si ces modifications concernent plusieurs lignes de la table).

– *Ils sont utiles si des opérations de groupe doivent être réalisées (comme le calcul d'une moyenne, d'une somme totale, d'un compteur, ...).*

Pour des raisons de performance, il est préférable d'employer les triggers de table plutôt que les triggers lignes.

Déclencheurs/Triggers (4/13)

Syntaxe

```
CREATE [OR REPLACE] TRIGGER NomTrigger  
{ BEFORE | AFTER} liste_Instructions  
ON nom_de_la_Table  
[FOR EACH ROW]  
[ WHEN condition]  
BLOC PL/SQL
```

Déclencheurs/Triggers (5/13)

L'option **BEFORE /AFTER** indique le moment du déclenchement du trigger.

Les **instructions SQL** (Par exemple : INSERT OR UPDATE OR DELETE) peuvent être toutes présentes comme on peut en avoir juste une.

Pour un UPDATE, on peut spécifier une liste de colonnes (UPDATE OF listeAttributs). Dans ce cas, le trigger ne se déclenchera que s'il porte sur l'une des colonnes précisées dans la liste.

FOR EACH ROW est utilisée pour les triggers de niveau ligne.

WHEN condition : le trigger est déclenché si la condition est vraie pour chaque ligne.

Déclencheurs/Triggers (6/13)

Conditions de déclenchement d'un trigger

Le trigger se déclenche lorsqu'un événement précis survient : BEFORE UPDATE, AFTER DELETE, AFTER INSERT, ...

Ces événements sont importants car ils définissent **le moment d'exécution du trigger**.

Ainsi, lorsqu'un trigger **BEFORE DELETE** est programmé, il sera exécuté *juste avant la suppression d'un nouvel* élément dans la table.

Déclencheurs/Triggers (7/13)

Si plusieurs triggers sont présents pour une même table, l'ordre d'activation est :

1. BEFORE niveau table
2. BEFORE niveau ligne, aussi souvent que de lignes concernées
3. AFTER niveau ligne, aussi souvent que de lignes concernées
4. AFTER niveau table

Déclencheurs/Triggers (8/13)

Exemple1 : Trigger de table

Ecrire un déclencheur PL/SQL permettant d'afficher un message d'erreur « Ne supprimez pas de lignes dans la table affectation » si une opération de suppression des lignes de la table affectation est demandée.

Effectuer, par la suite, les instructions suivantes:

select count() from affectation;*

delete from affectation where numpersaff = 7477;

select count() from affectation;*

Déclencheurs/Triggers (9/13)

Au niveau de l'instruction **FOR EACH ROW**, il est possible avant la modification de chaque ligne, de lire l'ancienne ligne et la nouvelle ligne par l'intermédiaire de deux variables structurées :

:OLD.nomAttribut : correspond à la valeur avant la transaction UPDATE ou DELETE

:NEW.nomAttribut : correspond à la valeur après la transaction UPDATE ou INSERT

Remarque : lorsque les variables OLD et NEW sont utilisées dans la partie **WHEN condition**, il ne faut pas utiliser les ":"

Déclencheurs/Triggers (10/13)

Exemple de trigger sur Delete

Ecrire un déclencheur PL/SQL «msg_supp » permettant d'afficher le message « vous avez supprimé la personne numéro ...» si une opération de suppression d'une ligne dans la table affectation est effectuée.

Vérifier le déclenchement du trigger

Déclencheurs/Triggers (11/13)

Exemple de trigger sur INSERT

Ecrire un déclencheur PL/SQL «ctrl_insertion» permettant d'afficher un message d'erreur « on ne peut pas avoir un employé embauché après cette date» si une opération d'insertion d'une ligne dans la table personne contenant une date d'embauche > date_du_jour est effectuée.

Insérer une ligne contenant une date > date_du_jour pour vérifier le déclenchement du trigger

Déclencheurs/Triggers (12/13)

Déclencheur sur conditions multiples

Lorsqu'un trigger porte sur les opérations LMD, des prédicats peuvent être ajoutés dans le code, pour indiquer les opérations de déclenchement : Inserting, Updating, Deleting

Syntaxe

```
CREATE TRIGGER ...  
BEFORE/AFTER INSERT OR UPDATE OR DELETE ON nom_Table  
.....  
BEGIN  
.....  
IF INSERTING THEN ..... END IF;  
IF UPDATING THEN ..... END IF;  
IF DELETING THEN ..... END IF;  
.....  
END;
```

Déclencheurs/Triggers (13/13)

Exemple

Ecrire un déclencheur PL/SQL « msg_operations » permettant d'indiquer par un message pour chaque opération effectuée si c'est une opération d'insertion, de modification ou de suppression.